

1. What If There Were No Operating System? Imagine a computer without an operating system. Identify the problems a user would face and investigate how OS features solve them. Demonstrate at least three features using Linux and present a “Without OS vs With OS” comparison.

An operating system is the software that connects the user and the computer hardware. It takes care of important activities such as CPU management, memory management, file management, process management, and communication with hardware.

Imagine switching on a computer that has no operating system. The hardware can still start and perform its basic self-check through BIOS/UEFI, but there would be no operating system available to load programs, organize storage, manage memory, or provide a convenient way for the user to interact with the computer.

Challenges of a computer without an OS and their solutions:

A. Lack of CPU and Program Management

- Problem: Programs would have to manage CPU time themselves, making multitasking difficult.
- OS Solution: The OS schedules processes and allocates CPU time between them.

B. Lack of File and Storage Management

- Problem: Data would not be organized into proper files and folders, making it difficult to store and retrieve information.
- OS Solution: The OS provides a file system to create, organize, access, and manage files.

C. Lack of Memory Management

- Problem: Programs could interfere with each other's memory, causing errors or crashes.
- OS Solution: The OS allocates memory to programs and keeps track of its usage.

D. Lack of Hardware Management

- Problem: Applications would need to communicate directly with devices such as keyboards, printers, and network cards.
- OS Solution: The OS acts as an interface between applications and hardware.

E. Lack of Security and Access Control

- Problem: There would be no proper system of users, passwords, or permissions to protect data.
- OS Solution: The OS provides user accounts, authentication, and file permissions.

F. Lack of a Standard Environment for Running Programs

- Problem: Users would need detailed knowledge of the hardware to load and run applications.
- OS Solution: The OS provides a standard environment in which applications can be loaded and executed.

The three important features of Linux are:

A. CPU and Process Management

Linux manages the processor by controlling and scheduling processes. It gives different processes CPU time so that multiple programs can operate without the user having to manually control the processor.

The ps command can be used to view running processes:

ps

```
(praneetha@Praneetha)-[~/Desktop]
$ ps
  PID TTY          TIME CMD
 2322 pts/0        00:00:00 zsh
 3677 pts/0        00:00:00 ps
```

B. Memory Management

Linux keeps track of the computer's RAM and shows how much memory is being used and how much is available.

The following command can be used:

free -h

```
(praneetha@Praneetha)-[~/Desktop]
$ free -h
             total        used        free      shared  buff/cache   available
Mem:        3.8Gi        898Mi        2.1Gi         17Mi         1.1Gi        2.9Gi
Swap:        2.1Gi          0B        2.1Gi
```

C. File Management

Linux provides a file system that allows users to create, access, modify, and manage files and directories without directly dealing with the physical storage device.

```
(praneetha@Praneetha)-[~/Desktop]
$ mkdir assignment_1

(praneetha@Praneetha)-[~/Desktop]
$ cd assignment_1

(praneetha@Praneetha)-[~/Desktop/assignment_1]
$ touch assignment.txt

(praneetha@Praneetha)-[~/Desktop/assignment_1]
$ ls
assignment.txt
```

Here, mkdir creates a directory, cd enters the directory, touch creates a file, and ls displays its contents. The operating system keeps track of where the file is stored.

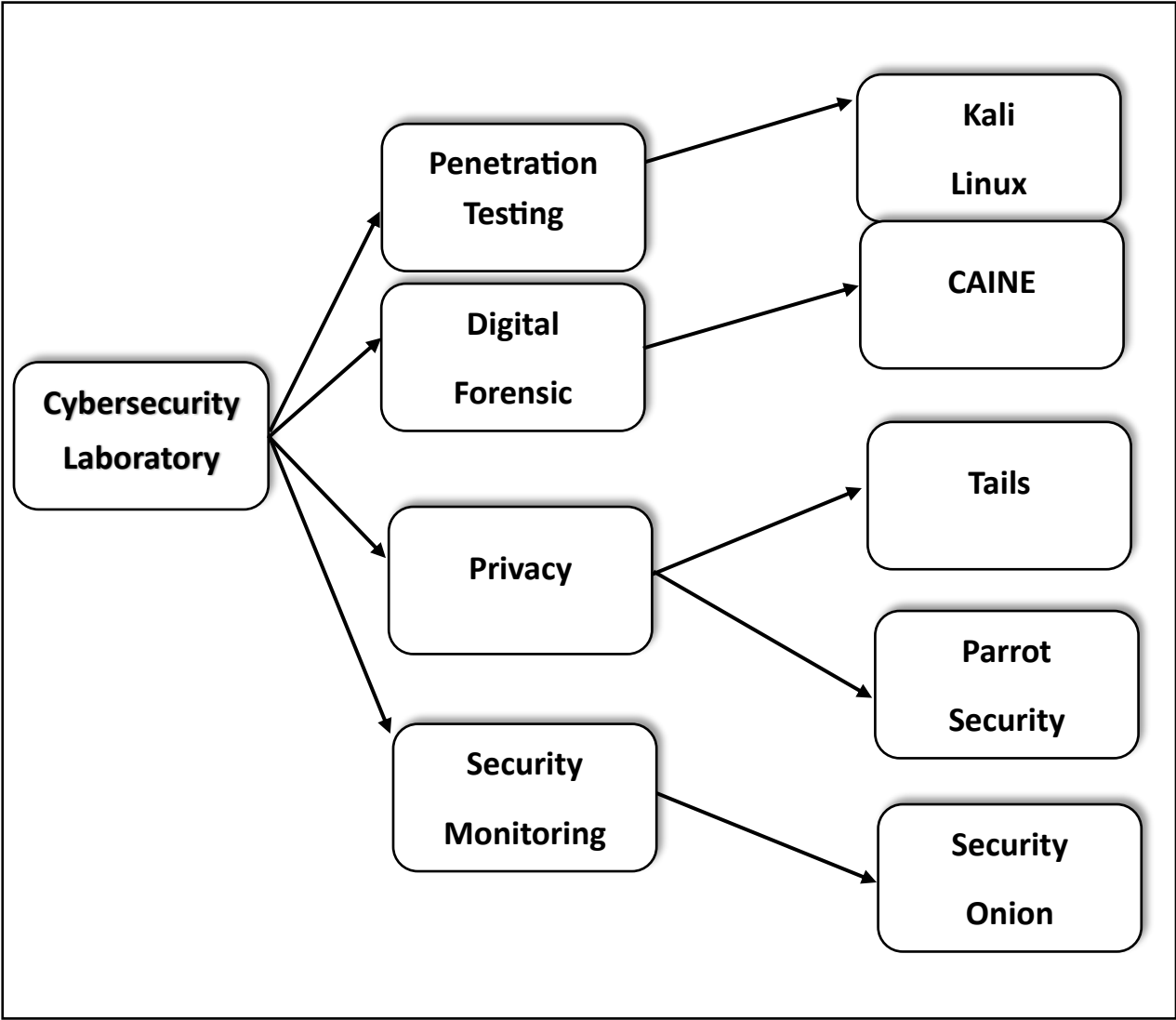
Functions	With OS	Without OS
CPU	Each program would have to handle the CPU on its own.	Schedules processes and distributes CPU time.
Memory	Programs would directly manage RAM, increasing the chance of conflicts.	Allocates memory and keeps track of its usage.
Files	Data would be difficult to organize and locate.	Provides files, directories, and commands to manage them.

Hardware	Applications would have to communicate directly with hardware.	Provides an interface and drivers between software and hardware.
Security	There would be little or no convenient access control.	User accounts, passwords, and permissions provide protection.
Running programs	Running several programs together would be difficult.	Supports multitasking and process management.
Ease of use	Users would need detailed knowledge of computer hardware.	Users can interact through a terminal or graphical interface.

2. Imagine your team is setting up a cybersecurity laboratory. Explore at least five Linux distributions relevant to cybersecurity and recommend suitable distributions for different roles such as penetration testing, digital forensics, privacy, and security monitoring. Present your findings creatively as a comparison chart, poster, infographic, or “OS selection guide.”

Linux Distribution	Suitable Role	Main Features	Advantages	Limitations
Kali Linux	Penetration Testing	Nmap, Metasploit, Wireshark and other security tools	Large collection of preinstalled penetration-testing tools	Can be complex for beginners
CAINE	Digital Forensics	Disk imaging, file recovery, evidence analysis	Provides many forensic tools in one environment	Requires knowledge of forensic techniques
Parrot Security OS	Privacy & Security Testing	Security tools, encryption and secure browsing	Lightweight and can also be used for everyday tasks	Less focused on anonymity than Tails
Tails	Privacy & Anonymity	Privacy-focused environment and encrypted storage	Designed specifically for privacy and anonymity	Some applications may have limited functionality
Security Onion	Security Monitoring	Network monitoring, intrusion detection.	Designed specifically for detecting and investigating security threats	Initial setup can be complex

- **Kali Linux - Penetration Testing**
Kali Linux is suitable for the penetration-testing role in the laboratory. It includes a large collection of security tools that can be used to identify and test vulnerabilities in controlled systems.
- **CAINE - Digital Forensics**
It is designed for digital forensic investigations. It provides tools that can be used to examine digital evidence after a security incident.
- **Parrot Security OS - Privacy and Security Testing**
Parrot Security OS combines cybersecurity tools with privacy-related features. It is also lightweight and can be used for normal everyday activities in addition to security testing.
- **Tails - Privacy and Anonymity**
Tails is specifically designed for privacy and anonymity. It can run from a USB drive and uses the Tor network to route internet traffic.
- **Security Onion - Security Monitoring**
Security Onion is intended for the defensive side of cybersecurity. It is used to monitor networks and identify suspicious activity rather than primarily performing attacks.



3. Exit status:

Execute at least five Linux commands that produce both successful and unsuccessful results. Investigate their exit status using '\$?'.

Create a simple visual/table showing Command → Result Exit Status → Meaning.

What can a script learn from an exit status?

When a Linux command finishes, it returns a number to indicate how the command ended. This number is called the exit status.

The special variable:

\$? can be used immediately after a command to check its exit status.

0: Command completed successfully

Non-zero: Command encountered an error or did not complete successfully

A. mkdir

- Successful execution

```
(praneetha@Praneetha)-[~/Desktop]
$ mkdir Linux

(praneetha@Praneetha)-[~/Desktop]
$ echo $?
0
```

The directory was created successfully.

- Unsuccessful execution

```
(praneetha@Praneetha)-[~/Desktop]
$ ls
assignment_1  Linux  MSIS

(praneetha@Praneetha)-[~/Desktop]
$ mkdir MSIS
mkdir: cannot create directory 'MSIS': File exists

(praneetha@Praneetha)-[~/Desktop]
$ echo $?
1
```

The command failed because the directory already existed.

B. ls

- Successful execution

```
(praneetha@Praneetha)-[~/Desktop]
$ ls
assignment_1  Linux  MSIS

(praneetha@Praneetha)-[~/Desktop]
$ echo $?
0
```

The contents of the current directory were displayed successfully.

- Unsuccessful execution

```
(praneetha@Praneetha)-[~/Desktop]
$ ls text
ls: cannot access 'text': No such file or directory

(praneetha@Praneetha)-[~/Desktop]
$ echo $?
2
```

The command failed because text does not exist.

C. date

- Successful execution

```
(praneetha@Praneetha)-[~/Desktop]
$ date
Thursday 13 August 2026 09:16:44 PM IST
assignment 1

(praneetha@Praneetha)-[~/Desktop]
$ echo $?
0
```

The current date and time were displayed successfully.

- Unsuccessful execution

```
(praneetha@Praneetha)-[~/Desktop]
$ date -d
date: option requires an argument -- 'd'
Try 'date --help' for more information.

(praneetha@Praneetha)-[~/Desktop]
$ echo $?
1
```

The command failed because the -d option was used without the required argument.

D. cd

- Successful execution

```
(praneetha@Praneetha)-[~/Desktop]
$ cd Linux

(praneetha@Praneetha)-[~/Desktop/Linux]
$ echo $?
0
```

The directory was changed successfully.

- Unsuccessful execution

```
(praneetha@Praneetha)-[~/Desktop/Linux]
$ cd Lab
cd: no such file or directory: Lab

(praneetha@Praneetha)-[~/Desktop/Linux]
$ echo $?
1
```

The command failed because the requested directory was not present.

E. Touch

- Successful execution

```
(praneetha@Praneetha)-[~/Desktop/Linux]
$ touch hello.txt

(praneetha@Praneetha)-[~/Desktop/Linux]
$ echo $?
0
```

The file was created successfully.

- Unsuccessful execution

```
(praneetha@Praneetha)-[~/Desktop/Linux]
$ touch /root/test.txt
touch: cannot touch '/root/test.txt': Permission denied

(praneetha@Praneetha)-[~/Desktop/Linux]
$ echo $?
1
```

The command failed because the specified folder did not exist.

Command	Exit Status	Meaning
mkdir Linux	0	Directory was created successfully
mkdir MSIS	1	Directory already existed
ls	0	Directory contents were displayed
ls text	2	text could not be found
date	0	Date and time were displayed
date -d	1	Required argument was missing
cd Linux	0	Successfully changed to the directory
cd Lab	1	Directory did not exist
touch hello.txt	0	File was created successfully
touch /root/test.txt	1	The specified directory does not exist.

What can a script learn from an exit status?

A script can use the exit status to know whether a command worked or failed.

- 0: The command was successful.
- Non-zero value: The command failed or encountered an error.

The script can then decide what to do next, such as continue if the command succeeded or show an error message if it failed.